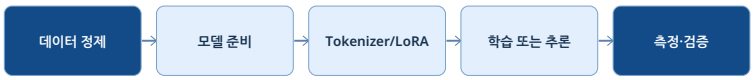
2026 보수교육 · 60분 실습 · 원본 교육과정 기준 · 적대적 검증 개선판 (2026-11-05)



📘 06차시 — LLM 모델의 파인튜닝 전후 비교 테스트

교육시간: 1일차 16:00–17:00 (60분) **실습 모델:** `Qwen/Qwen2.5-0.5B-Instruct` (제작 선택) **검증일:** 2026-11-04

01 | 개요·진행 일정

과제

기준 모델과 실제 학습된 LoRA 어댑터를 동일 조건에서 추론하여 결과를 기록하고, 성능 향상 주장의 근거를 판단한다.

60분 운영

0~10분: 비교 설계 / 10~20분: 데이터·환경 준비 / 20~40분: 코드 실행 / 40~50분: 결과 분석 / 50~60분: 평가·정리.

학습 성과

① 공정 비교 조건을 설명한다. ② 기준선 추론 결과를 저장한다. ③ 학습 어댑터가 있으면 동일 평가로 비교한다. ④ 실제 측정과 예시를 구분한다.

중요

이번 차시는 1일차 마지막 차시이다. 04차시는 LoRA 구조를 확인했을 뿐 학습 완료를 보장하지 않는다. 학습된 어댑터가 없다면 학습 전후 성능 비교는 성립하지 않는다.

---

02 | 배경·선수지식

기준 모델

사전학습 또는 지시문 튜닝된 원본 모델. 이번 예제에서는 Qwen/Qwen2.5-0.5B-Instruct를 선택한다. 과정안내서 지정 모델이 아닌 제작 선택이다.

학습 어댑터

학습을 통해 변경된 LoRA 행렬을 저장한 디렉터리. adapter_config.json과 어댑터 가중치 파일 등이 있어야 한다. 단순 get_peft_model() 호출은 학습이 아니다.

비교의 최소 요건

같은 질문·채팅 템플릿·토크나이저·생성 옵션·하드웨어를 유지한다. 검증 데이터는 학습 데이터와 분리한다.

주의

정답률이 상승해도 다른 질문에서 악화될 수 있다. 응답 시간은 초기 모델 적재·GPU 워밍업·메모리 상태에 민감하다.

03 | 핵심 원리와 평가 척도

평가 구조

평가 입력 X → 기준 모델 f_base(X) / 학습 모델 f_adapter(X) → 출력 비교. 같은 입력에 대한 출력 차이가 곧 품질 향상을 뜻하지 않는다.

정확성

정답이 명확한 문항은 사전에 정의한 채점 기준으로 점수를 계산한다. 자유서술형은 근거성·정확성·누락·환각 여부를 별도 판정한다.

속도

각 문항 생성 소요 시간(초)을 기록한다. 모델 최초 실행의 워밍업 비용과 실험 반복 편차를 따로 기록한다.

재현성

모델 식별자·어댑터 경로·라이브러리 버전·GPU 이름·평가 질문·출력 JSON을 함께 보존한다.

04 | 모델 비교 구조도

처리 흐름

평가 CSV → 동일 질문/템플릿 → [기준 모델] → base 응답 → 결과 JSON ↘ [기준 모델 + 학습된 LoRA] → adapter 응답 ↗

판정 흐름

결과 JSON → 문항별 답변 검토 → 정확성·근거성 채점 → 차이 분석 → 개선 여부 판정.

데이터 분리

학습 데이터와 평가 데이터는 분리해야 한다. 질문을 외워 답하는 현상을 성능 향상으로 오인하지 않도록 중복·유사 중복을 점검한다.

통제 항목

프롬프트, max_new_tokens, do_sample=False, 실행 환경, 입력 순서. 어댑터를 바꾸는 것 외의 조건을 가능한 한 고정한다.

---

05 | 준비·설치·파일 구성

파일

lesson06_compare.py : 기준/어댑터 추론과 JSON 저장 lesson06_eval.csv : id,question,reference 열의 평가 데이터

Ubuntu 명령어

기존 가상환경 활성화

source .venv/bin/activate

설치 라이브러리 버전 확인

python -m pip show torch transformers peft

문법 검사

python -m py_compile lesson06_compare.py

의존성

torch, transformers, peft(어댑터 사용 시), CUDA(선택). 1~5차시에서 구성한 가상환경을 재사용한다.

실행 전 확인

평가 CSV의 문항이 실제 학습 데이터에 포함되지 않았는지 점검하고, 어댑터가 실제 학습 결과인지 확인한다.

---

06 | 실습 코드·주석 해설

핵심 코드 1

tokenizer = AutoTokenizer.from_pretrained(MODEL_ID) model = AutoModelForCausalLM.from_pretrained(MODEL_ID, dtype=dtype) model = model.to(device).eval()

핵심 코드 2

inputs = tokenizer.apply_chat_template(messages, tokenize=True, add_generation_prompt=True, return_tensors="pt", return_dict=True) outputs = model.generate(**inputs, max_new_tokens=96, do_sample=False)

핵심 코드 3

tuned = PeftModel.from_pretrained(model, args.adapter).eval()

학습된 어댑터가 있을 때만 적용. 무학습 어댑터를 비교 결과로 사용하지 않는다.

전체 코드

동봉된 lesson06_compare.py에 입력 검증·추론·실행시간 측정·JSON 저장·오류 메시지를 포함한다. 수업에서는 파일 전체를 열어 주석을 읽고 실행한다.

---

07 | 실행 결과와 검증

기준선 실행

python lesson06_compare.py --csv lesson06_eval.csv --output base.json

예상 메시지

결과 저장: base.json, 레코드: 3 주의: 학습 어댑터 미지정. 학습 전후 성능 비교가 아니라 기준선만 기록했습니다.

어댑터 비교

python lesson06_compare.py --csv lesson06_eval.csv --adapter ./trained_adapter --output compare.json

출력 형식

JSON 레코드에 id, question, reference, variant, answer, seconds를 저장한다. 어댑터 비교 성공 시 문항 3개 × 2변형 → 6개 레코드.

검증 경계

위 메시지는 예상 출력 형식이다. 실제 모델 다운로드·GPU 추론·학습 어댑터 품질은 이 제작 환경에서 검증하지 않았다.

---

08 | 따라하기 실습

실습 A — 기준선

① CSV의 3개 질문을 확인한다. ② 기준선 명령을 실행한다. ③ base.json을 열어 출력과 실행시간을 확인한다.

실습 B — 학습 모델

① 학습된 어댑터 폴더를 준비한다. ② --adapter 인자를 추가한다. ③ compare.json에서 같은 id의 base/adapter 응답을 짝지어 비교한다.

실습 C — 채점

각 문항을 정확/부분정확/오답으로 채점하고, 근거 없는 내용 여부를 별도 기록한다. 참고 정답(reference)은 자동 정답 판정 코드가 아니라 사람이 확인할 기준 문장이다.

시간 배분

10분 준비, 15분 기준선, 15분 어댑터 비교(없으면 결과표 설계), 10분 평가, 10분 정리.

09 | 오류 대처·확장 실습

모델 다운로드 실패

네트워크, 디스크 용량, 모델 ID 및 접근 권한을 확인한다.

CUDA 메모리 부족

다른 GPU 작업 종료, 더 작은 모델·CPU 실행 검토. CPU는 매우 느릴 수 있다.

어댑터 미발견

--adapter 경로 및 adapter_config.json 확인. 04차시의 초기화만 수행한 모델은 학습된 결과가 아니다.

비교 왜곡

생성 옵션 변경·서로 다른 질문·평가 데이터 누출·워밍업 누락을 점검한다.

확장

동일 문항을 여러 차례 반복해 중앙값 시간을 계산하고, 채점자 간 평가 차이를 기록한다.

---

10 | 평가·체크리스트·출처

확인 문제

Q1. LoRA 초기화만 수행한 모델을 학습 후 모델이라 할 수 있는가? (아니오) Q2. 비교 시 고정해야 할 3가지는? (질문·생성옵션·환경 등) Q3. 응답 시간이 1회 짧아졌다면 성능 향상인가? (반복 측정 필요)

완료 체크

☐ CSV 형식 확인 ☐ 기준선 결과 저장 ☐ 어댑터 학습 여부 확인 ☐ 동일 조건 비교 ☐ 평가표 작성 ☐ 미검증 항목 표시

공식 참고

Hugging Face Transformers 생성: https://huggingface.co/docs/transformers/main_classes/text_generation Hugging Face PEFT: https://huggingface.co/docs/peft/index Qwen2.5-0.5B-Instruct: https://huggingface.co/Qwen/Qwen2.5-0.5B-Instruct

검증일·범위

2026-11-04. 공식 문서 주소를 참고용으로 수록했으며 최신 API를 온라인에서 실시간 대조한 것은 아니다. 코드 문법과 PDF 페이지/한글 추출은 로컬 검사한다.

다음 차시

07차시: 파인튜닝에 사용할 데이터셋 전처리. 학습/검증 분리와 대화형 데이터 형식을 다룬다.

전체 Python 실습 코드

```
"""06차시: 기준 모델과 학습 완료된 LoRA 어댑터의 공정한 추론 비교."""
import argparse
import csv
import json
import time
from pathlib import Path

import torch
from transformers import AutoModelForCausalLM, AutoTokenizer

MODEL_ID = "Qwen/Qwen2.5-0.5B-Instruct"  # 교육용 선택 모델


def load_questions(path):
    with open(path, encoding="utf-8") as file:
        rows = list(csv.DictReader(file))
    if not rows or not {"id", "question", "reference"}.issubset(rows[0]):
        raise ValueError("CSV에는 id, question, reference 열과 1개 이상의 행이 필요합니다.")
    return rows


def generate(model, tokenizer, question, max_new_tokens):
    messages = [{"role": "user", "content": question}]
    # (변경) 템플릿을 바로 토큰화하여 특수 토큰 중복을 방지한다.
    inputs = tokenizer.apply_chat_template(
        messages, tokenize=True, add_generation_prompt=True,
        return_tensors="pt", return_dict=True
    )
    device = next(model.parameters()).device
    inputs = {key: value.to(device) for key, value in inputs.items()}
    if torch.cuda.is_available():
        torch.cuda.synchronize()
    started = time.perf_counter()
    with torch.inference_mode():
        outputs = model.generate(
            **inputs, max_new_tokens=max_new_tokens,
            do_sample=False, pad_token_id=tokenizer.eos_token_id
        )
    if torch.cuda.is_available():
        torch.cuda.synchronize()
    elapsed = time.perf_counter() - started
    new_tokens = outputs[0, inputs["input_ids"].shape[1]:]
    return tokenizer.decode(new_tokens, skip_special_tokens=True).strip(), elapsed
```

```
def main():
    parser = argparse.ArgumentParser()
    parser.add_argument("--csv", default="lesson06_eval.csv")
    parser.add_argument("--adapter", default=None,
                        help="학습 완료된 PEFT LoRA 어댑터 폴더; 생략하면 기준 모델만 실행")
    parser.add_argument("--output", default="lesson06_results.json")
    parser.add_argument("--max-new-tokens", type=int, default=96)
    args = parser.parse_args()
    questions = load_questions(args.csv)
    tokenizer = AutoTokenizer.from_pretrained(MODEL_ID)
    # CPU 환경에서도 동작하도록 dtype/device 선택. GPU 모델 메모리 상황 확인 필수.
    dtype = torch.float16 if torch.cuda.is_available() else torch.float32
    model = AutoModelForCausalLM.from_pretrained(MODEL_ID, dtype=dtype)
    model = model.to("cuda" if torch.cuda.is_available() else "cpu").eval()
    records = []
    for row in questions:
        answer, seconds = generate(model, tokenizer, row["question"], args.max_new_tokens)
        records.append({**row, "variant": "base", "answer": answer,
                        "seconds": round(seconds, 4)})
    if args.adapter:
        from peft import PeftModel
        if not Path(args.adapter).is_dir():
            raise FileNotFoundError("학습된 LoRA 어댑터 폴더를 찾을 수 없습니다.")
        tuned = PeftModel.from_pretrained(model, args.adapter).eval()
        for row in questions:
            answer, seconds = generate(tuned, tokenizer, row["question"], args.max_new_tokens)
            records.append({**row, "variant": "adapter", "answer": answer,
                            "seconds": round(seconds, 4)})
    Path(args.output).write_text(json.dumps(records, ensure_ascii=False, indent=2),
                                 encoding="utf-8")
    print(f"결과 저장: {args.output}, 레코드: {len(records)}")
    if not args.adapter:
        print("주의: 학습 어댑터 미지정. 학습 전후 성능 비교가 아니라 기준선만 기록했습니다.")


if __name__ == "__main__":
    main()
```

적대적 검증·개선 기록 (2026-11-05)

반영: 실습 코드의 템플릿 중복 토큰화 위험을 수정하고 dtype 인자를 최신 형식으로 정리했습니다.

검증 구분: 코드 구문·입력 데이터·PDF 및 HTML 생성과 검증은 수행했으나 실제 모델 다운로드·GPU 학습·추론 품질·Ollama 배포는 별도의 실물 검증이 필요합니다.

재현성: 교육용 5스텝 학습의 손실 감소나 정확도 향상을 보장하지 않습니다. 모델 ID, 데이터 버전, 장치·패키지 버전, 난수 시드, 생성 설정을 기록해야 합니다.

공식 근거: SFTTrainer, PEFT 연동, 템플릿, PEFT 모델 저장, 양자화, Ollama GGUF (2026-11-05 확인)